

Kiro Hackathon: IAM Trouble Game

https://gemini.google.com/app/1f74048e11857b01

User prompt: i have shared project idea for a hackthon that we were discussing in earlier conversation which we have to build using Kiro in 4 hr hackthon,, do you remember..??

Response: I have our context right here,, and **IAM In Trouble** is an absolute standout concept! Turning the real-world terror of an unexpected cloud bill into a gamified, risk-free terminal escape room is both hilarious and incredibly practical for developers.

Since you have a strict 4-hour window to pull this off using Kiro, utilizing its spec-driven agentic development features will be the key to executing this on time.

⚡ 4-Hour Kiro Sprint Strategy

Here is how you can adapt your execution plan into a complete Kiro workflow, ensuring you have plenty of room for AI iteration and manual debugging:

Sprint Phase	Kiro Action	Time Allocation	Focus Area
1. Setup & Specs	@quickstart & @prime	30 mins	Initialize LocalStack, map your pitch into product.md.
2. Backend Engine	@plan-feature -> @execute	1.5 hours	Provision EC2/S3 using Python (boto3), write verify.py logic.
3. Terminal Theater	@execute	1 hour	Generate Bash ASCII art and the live billing ticker loop.
4. Polish & Demo	@code-review & Test	1 hour	Squash bugs, rehearse the 2:00 AM PagerDuty pitch!

🔧 Execution Strategy

- **Spec-Driven Leverage:** Kiro thrives on context. You can drop your "Core Features" and "Levels" directly into the project's steering documents. The agent will read that context and immediately understand the success criteria for the game.
 - **The Backend Logic:** Tackling the JSON policy parsing for the custom IAM evaluator will be a seamless extension of your work with backend authorization frameworks. You can guide Kiro to quickly generate the validation scripts for the policy changes.
 - **Terminal Environment:** Whipping up the Bash ASCII theater and the game loop triggers will feel just like writing your usual custom WSL automation scripts.
 - **Time Management:** Leading a hackathon team at Amalthea demonstrated that scope creep is the ultimate project killer. Your planned timeline smartly builds in a generous 2-hour buffer to handle any inevitable Docker or LocalStack networking quirks before the final buzzer.
- Hackathon Pro-Tip:** Ensure Kiro configures the "Bankrupt Counter" as an asynchronous background process in Bash. It shouldn't block the user's CLI inputs—you want them actively sweating over the ticking money while they frantically type their commands!

Are you ready to start drafting the exact @plan-feature prompts for the backend engine, or should we flesh out the Python validation logic for the levels first..??

User prompt: give me step-by-step roadmap to properly build it and implement it..!!

Response: Let's break this down into a highly actionable, step-by-step roadmap so you can hit the ground running the second the clock starts. Executing this within a 4-hour hackathon window means keeping the scope perfectly locked and leaning heavily on Kiro's agentic workflows to generate the boilerplate code.

Running this directly within a WSL environment will ensure the Bash scripts, custom terminal loops, and standard AWS CLI commands interact flawlessly without any pathing issues.

Here is your exact attack plan to build **IAM In Trouble**:

Phase 1: Setup & Environment Prep (10:00 AM)

Your priority is getting the core infrastructure breathing and feeding Kiro the project rules.

- **Initialize LocalStack:** Warm up your cloud environment by running the Docker command `docker run -d -p 4566:4566 localstack/localstack`. This provides the free community edition backend.
- **Prime Kiro:** Drop your exact "Cloud Breach Blueprint.md" text into Kiro to generate the `requirements.md` and `tasks.md` specs.
- **Configure the CLI:** Ensure the standard AWS CLI is installed and configured to route commands locally using the `--endpoint-url=http://localhost:4566` flag.

Phase 2: The Python Backend Engine (10:45 AM)

This sprint focuses entirely on writing the Python 3 and boto3 scripts to spawn the puzzle and check the answers.

- **Build setup.py:** Instruct Kiro to write a Python script that provisions the "hacked" state:
 - **Level 1:** Spawn a `p4d.24xlarge` EC2 instance to act as the Dogecoin mining rig.

- **Level 2:** Create an S3 bucket named `customer-passwords-do-not-share` and explicitly set it to Public Read.
- **Level 3:** Generate a local `attacker_policy.json` file containing an explicit Deny rule.
- **Build `verify.py`:** This script will validate the player's actions:
 - Have it query LocalStack to verify the EC2 instance state is completely terminated.
 - Have it check the S3 bucket ACLs to confirm public-read access is safely revoked.
 - Implement a custom Python policy evaluator that parses the edited JSON file to ensure the malicious Deny block has been deleted.

Phase 3: The Terminal Theater (12:30 PM)

This is where you build the gamified experience using Bash and ASCII formatting. No web UI bloat—just dramatic terminal visuals.

- **The UI Shell:** Create `./start_game.sh` to act as the opening screen. Prompt Kiro to generate dramatic retro ASCII text and neon colors for the PagerDuty alert UI.
- **The Bankrupt Counter:** Write a background Bash loop that displays a live cost ticker in the top right of the terminal. It needs to tick up fake money lost (e.g., \$32.50... \$48.00...) continuously while the player types.
- **The Briefing:** Create `./briefing.sh` to dump the narrative text about the leaked API key and the rogue crypto-miners.
- **The Wrapper:** Create the `./verify.sh` script that takes arguments (e.g., `./verify.sh 1`) and passes them to your underlying `verify.py` engine to trigger the success/fail logic for each level.

Phase 4: Polish & Test (1:00 PM)

To maintain momentum and avoid feature creep, freeze development here and strictly test the three levels.

- Run `./start_game.sh` and ensure the background money ticker doesn't block standard input.
- Play through Level 1 by running `aws --endpoint-url=http://localhost:4566 ec2 describe-instances`, grabbing the ID, and executing the `terminate-instances` command.
- Lock down the S3 bucket using `s3api put-bucket-acl` with the `--acl private` flag.
- Open `attacker_policy.json` in nano, delete the explicit deny, save, and ensure `./verify.sh 3` triggers the victory screen.

Phase 5: The Hackathon Demo (1:30 PM)

- **Rehearse the Pitch:** Hook the judges immediately with the terrifying 2:00 AM PagerDuty narrative.
- **Live Execution:** Live-execute the authentic CLI commands targeting LocalStack to prove the application delivers 100% real AWS CLI muscle memory with \$0 real-world risk.

Which sprint phase do you want to tackle with Kiro first—should we draft the exact prompts for the `setup.py` boto3 logic, or start visualizing the Bash terminal loops..??

User prompt: Give me steps for implementing/building Phase 1 with detailed explanation about how to build it and how it's working

Response: Here is the detailed, step-by-step technical breakdown of how to build **Phase 1: Setup & Environment Prep**, including exactly how to execute it and what is happening under the hood.

Step 1: Initialize the LocalStack Backend

The first objective is to spin up the mock AWS environment. LocalStack operates by running a localized API gateway that intercepts and mocks standard AWS API calls so you can build locally without incurring cloud costs.

How to build it: Open your terminal and execute the following command to pull and run the LocalStack image:

```
docker run -d -p 4566:4566 localstack/localstack
```

- You are utilizing the Docker platform alongside the LocalStack Community Free Edition for the cloud backend.

How it works under the hood:

- `docker run`: Instructs the Docker daemon to pull the LocalStack container image from Docker Hub (if it is not already cached locally) and spin it up.
- `-d (Detached mode)`: Runs the container in the background so your terminal remains free for Kiro commands and Bash scripts.
- `-p 4566:4566 (Port binding)`: Maps port 4566 on your local machine to port 4566 inside the container. This is the single API gateway port LocalStack uses to route all incoming requests (whether for EC2, S3, or IAM) to the correct mocked service.

Step 2: Configure the AWS CLI

Your game requires players to use the authentic AWS CLI. However, the standard CLI expects to communicate with real AWS servers using valid security credentials. You need to configure it to route local traffic to your Docker container instead.

How to build it: First, set up dummy credentials to bypass the AWS CLI's authentication checks by running `aws configure`:

```
AWS Access Key ID [None]: test
AWS Secret Access Key [None]: test
Default region name [None]: us-east-1
Default output format [None]: json
```

Next, whenever executing a command in the game, append the local endpoint flag. For example:

```
aws --endpoint-url=http://localhost:4566 ec2 describe-instances
```

- You will use authentic AWS CLI commands targeting LocalStack via the `--endpoint-url=http://localhost:4566` syntax.

How it works under the hood: LocalStack doesn't actually validate standard AWS credentials, but the `aws` CLI binary will refuse to run if it doesn't find credentials configured in your `~/.aws/credentials` file. Providing dummy values ("test") satisfies the CLI's internal checks. The `--endpoint-url` flag overrides the default AWS domain (e.g., `ec2.us-east-1.amazonaws.com`) and forces the CLI to fire its API requests directly at your LocalStack container running on port 4566.

Step 3: Prime Kiro with Specs

Because you are under a 4-hour hackathon time limit, you need Kiro to generate the boilerplate Python setup scripts and Bash logic. Kiro uses a "Spec-driven" architecture to plan features before writing code.

How to build it:

1. Open your project directory in your IDE and launch Kiro.
2. Start a new "Feature Spec" session.
3. Upload or paste the exact "Cloud Breach Blueprint.md" text into the chat.
4. Tell Kiro: *"Generate the requirements and technical design for Phase 2 and Phase 3 based on this blueprint."*

How it works under the hood: When Kiro processes your prompt, it won't just blindly spit out code. It leverages an automated multi-agent workflow to build a structured specification directory (`.kiro/specs/iam-in-trouble`).

- It generates a `requirements.md` to map out the levels and success criteria.
- It generates a `design.md` outlining the technical architecture (e.g., using `boto3` for Python validation).
- It populates a `tasks.md` file with a dependency graph of trackable execution steps. You can then trigger parallel execution in Kiro to rapidly generate `setup.py`, `verify.py`, and the Bash UI scripts simultaneously.

Now that the environment is fully prepped and Kiro is primed with the rules, are you ready to generate the specific Python `boto3` logic for spawning the hacked EC2 instance and S3 bucket in Phase 2?

User prompt: Give me steps for implementing/building Phase 2 with detailed explanation about how to build it and how it's working

Response: Here is the detailed, step-by-step technical breakdown of how to build **Phase 2: The Python Backend Engine**, focusing on writing the Python 3 and `boto3` scripts to simulate the attack and evaluate the player's responses.

Step 1: Connecting Python to LocalStack

To manipulate your mock AWS environment, both `setup.py` and `verify.py` must utilize the `boto3` library and explicitly point their clients to LocalStack.

The Boilerplate Code: Instruct Kiro to use the following structure at the top of both files:

```
import boto3
import json

# Route Boto3 traffic to LocalStack instead of real AWS servers
endpoint_url = "http://localhost:4566"
ec2_client = boto3.client("ec2", endpoint_url=endpoint_url, region_name="us-east-1", aws_access_key_id="test", aws_secret_access_key="test")
s3_client = boto3.client("s3", endpoint_url=endpoint_url, region_name="us-east-1", aws_access_key_id="test", aws_secret_access_key="test")
```

- You use `boto3` to configure the Python 3 clients with dummy credentials and target the API gateway on port 4566.

Step 2: Building setup.py (Simulating the Hack)

This script will be executed once at the start of the game to provision the broken state that the player must fix.

How to build it:

- **Level 1 (EC2):** Instruct Kiro to use `ec2_client.run_instances()` to spawn the rogue crypto-miner. Set the `InstanceType` to `p4d.24xlarge`.
- **Level 2 (S3):** Instruct Kiro to use `s3_client.create_bucket()` to generate a bucket named `customer-passwords-do-not-share`. Follow it with `s3_client.put_bucket_acl(Bucket='customer-passwords-do-not-share', ACL='public-read')` to intentionally leak the data.

- **Level 3 (IAM):** Instruct Kiro to use standard Python file I/O (`json.dump`) to create a local file named `attacker_policy.json`. Have it inject a JSON policy structure containing an explicit "Effect": "Deny" statement targeting a payroll database.

Step 3: Building `verify.py` (The Evaluator Engine)

This script acts as the game engine, checking the player's answers against LocalStack's current state. It will accept a level number as an argument from the Bash wrapper and run the corresponding check.

How to build it:

- **Level 1 Verification:** Instruct Kiro to use `ec2_client.describe_instances()` to retrieve the state of the `p4d.24xlarge` instance. The script must assert that the `InstanceState` is "terminated".
- **Level 2 Verification:** Instruct Kiro to call `s3_client.get_bucket_acl()` on the target bucket. It must parse the returned Grants list to verify that the `public-read` access is completely revoked.
- **Level 3 Verification:** Instruct Kiro to build a lightweight Python engine to evaluate the custom IAM logic. It must use `json.load()` to read the player-edited `attacker_policy.json`. The evaluator will loop through the "Statement" array to confirm the explicit Deny block on the payroll database has been deleted.

How it works under the hood

- **API Interception:** When `setup.py` or `verify.py` invokes a boto3 method like `describe_instances`, the HTTP request doesn't travel to AWS servers in `us-east-1`. The `endpoint_url` intercepts it and routes the JSON payload directly to the LocalStack Docker container running locally. LocalStack parses the request, updates its internal mock state, and returns an identical JSON response to your Python script.
- **Policy Evaluation:** By building a custom Python parser for Level 3, you bypass the need for a full IAM backend. You simply validate the JSON structure as standard Python dictionaries, making the explicit Deny check incredibly fast and reliable.

Should we draft the exact shell commands for Phase 3 so Kiro can start building the Bash loops and ASCII UI?

User prompt: Give me steps for implementing/building Phase 3 with detailed explanation about how to build it and how it's working

Response: Here is the detailed, step-by-step technical breakdown of how to build **Phase 3: The Terminal Theater**.

This sprint is entirely about building the gamified, high-stakes experience using Bash and ASCII formatting. The objective is to deliver immersive visual feedback with zero web UI bloat.

Step 1: Building the Bankrupt Counter (The Live Ticker)

This feature creates the sense of urgency by simulating the financial drain of an AWS breach.

- **How to build it:** Instruct Kiro to write a background Bash `while` loop that increments a fake money lost variable. The counter should update values sequentially (e.g., \$32.50... \$48.00...).
- **Terminal Placement:** Direct the agent to use standard ANSI escape codes within the loop to print this live ticker specifically in the top right of the terminal.
- **Execution:** The loop must be appended with an `&` symbol so it operates asynchronously, allowing the player to continue panic-typing commands without the prompt blocking their input.

Step 2: Designing the UI Shells (`start_game.sh` & `briefing.sh`)

These scripts set the stage and explain the rules before the player touches the cloud backend.

- **Building `start_game.sh`:** This acts as the opening screen. Instruct Kiro to use dramatic retro text and neon colors to generate the PagerDuty alert UI box.
- **Building `briefing.sh`:** This file executes the narrative dump. It must echo the story explaining that a leaked API key was pushed to a public GitHub repo. It should also explain the attacker's actions, specifically exposing customer secrets and running a Dogecoin mining rig.

Step 3: The Wrapper (`verify.sh`)

This script bridges the visual theater with your Python game engine.

- **How to build it:** Create a simple executable bash script that accepts level numbers as arguments (e.g., `./verify.sh 1`).
- **Integration:** Instruct Kiro to pass this argument directly to the `verify.py` script you built in Phase 2.
- **Victory State:** If the Python engine returns a success state, `verify.sh` will trigger the Game Over Screen. It must print an ASCII banner confirming the threat is neutralized, calculating the fake money saved (e.g., \$42,100), and granting the "Promoted to Senior SRE" status.

How it works under the hood

- **Terminal Positioning:** By leveraging standard ANSI escape sequences (like `\033[s` to save the cursor state, `\033[1;80f` to move to the top right coordinates, and `\033[u` to restore the cursor), the Bash script can continuously overwrite the ticking dollar amount in the corner. This runs incredibly smoothly within standard Linux terminal emulators and local WSL setups without disrupting the active command line prompt.

- **Process Backgrounding:** When you execute the ticker loop with `&`, Bash spawns it as a background subshell process. The terminal immediately returns control to the player. This is what allows the game to deliver 100% real AWS CLI muscle memory—the player is freely interacting with the authentic `aws` binary in the foreground while the timer aggressively counts up in the background.

Should we structure the exact ASCII border layouts for the "Game Over Screen" next, or would you prefer to review the specific `boto3` queries for the Level 1 EC2 checks first..??

User prompt: Give me steps for implementing/building Phase 4 with detailed explanation about how to build it and how it's working

Response: Phase 4: Polish & Test (1:00 PM)

Step 1: Validate Terminal Theater Mechanics Before testing the cloud resources, ensure the gamified UI elements run smoothly without breaking standard input.

- **Trigger the Game:** Run `./start_game.sh` to initialize the environment.
- **Test Asynchronous Execution:** Verify that the background "Bankrupt Counter" loop successfully updates the fake money lost in the top right corner without freezing your terminal prompt.
- **Process Cleanup Check:** Ensure that reaching a victory state properly kills the asynchronous Bash loop.

Step 2: Execute Level 1 Test (The Crypto Miner) Test the EC2 environment and verification logic against the LocalStack mock backend.

- **Query LocalStack:** Run the standard CLI command using `aws --endpoint-url=http://localhost:4566 ec2 describe-instances` to locate the suspicious `p4d.24xlarge` instance ID.
- **Terminate the Threat:** Execute `aws --endpoint-url=http://localhost:4566 ec2 terminate-instances --instance-ids i-xxxxx` to mock-delete the miner.
- **Verify State:** Execute `./verify.sh 1`. This must trigger the Python engine to assert that the LocalStack instance state registers as terminated.

Step 3: Execute Level 2 Test (Who Let The S3 Out?) Test the S3 bucket access restrictions.

- **Apply the Fix:** Use the CLI to lock down the compromised bucket by running `aws --endpoint-url=http://localhost:4566 s3api put-bucket-acl --bucket <bucket-name> --acl private`.
- **Verify State:** Execute `./verify.sh 2`. The underlying `boto3` script must evaluate the bucket ACLs to confirm that `public-read` access is securely revoked.

Step 4: Execute Level 3 Test (IAM Engine Evaluation) Test the custom Python JSON parser.

- **Apply the Fix:** Open `attacker_policy.json` locally using `nano`. Manually find the explicit Deny block, delete it, and save the file.
- **Verify State:** Execute `./verify.sh 3` to feed the edited JSON directly into your custom evaluator.
- **Trigger Victory Screen:** The evaluator must successfully verify that full access is restored and print the Game Over screen. This includes the "THREAT NEUTRALIZED" ASCII banner and the "Promoted to Senior SRE" status.

Should we establish the precise background process termination commands for the Bash loops, or would you like to prepare the Level 1 Python testing logic for Kiro..??

User prompt: Give me steps for implementing/building Phase 5 with detailed explanation about how to build it and how it's working

Response: Here is the strategic breakdown of how to build and execute **Phase 5: The Hackathon Demo (1:30 PM)**.

Because hackathon judges suffer from serious presentation fatigue, this phase is less about coding and entirely about structuring a compelling, high-energy technical performance.

Step 1: The PagerDuty Hook

You need to immediately hook the judges by dropping them right into the panic of the 2:00 AM story.

- **The Pitch:** Explain the terrifying scenario: a leaked API key on a public GitHub repo has allowed a hacker to compromise the AWS account. Emphasize that the attacker is currently exposing customer secrets and running a massive GPU instance to mine Dogecoin.
- **How it works (Psychologically):** Judges hear dozens of dry, feature-heavy pitches. By opening with the universally feared problem of waking up to a massive, unexpected cloud bill, you instantly validate the project's core purpose: giving developers a risk-free environment to practice without the threat of a ruined bank account.

Step 2: Initializing the Terminal Theater

Transition smoothly from the narrative hook straight into the visual gameplay.

- **The Execution:** Run `./start_game.sh` on the big screen. Let the dramatic retro text, neon colors, and ASCII banners speak for themselves.
- **How it works (Technically):** Because you built the "Bankrupt Counter" as an asynchronous Bash loop, the fake money lost (e.g., \$32.50... \$48.00...) will automatically start ticking up in the top right corner of the terminal. This live, visual drain of company funds creates

a visceral sense of urgency in the room without requiring a single piece of web UI bloat.

Step 3: Live Demo Execution (The Muscle Memory)

This is where you prove the technical viability of the engine. To ensure the demo doesn't crash due to a live-typing typo, firmly separate team roles: assign one person to be the dedicated "Demo Driver" at the keyboard, while you or another teammate acts as the Speaker delivering the pitch.

- **The Execution:** Have the Demo Driver live-execute the authentic CLI commands, such as `aws --endpoint-url=http://localhost:4566 ec2 describe-instances` to hunt down the crypto-miner.
- **How it works (Technically):** Live-executing these standard commands against your LocalStack container proves that your game delivers 100% real AWS CLI muscle memory. You are showing the judges in real-time that a junior dev can practice terminating rogue resources and modifying IAM rules with \$0 real-world risk.

Step 4: The Victory Screen & Wrap-Up

End the presentation on a definitive, highly visual high note.

- **The Execution:** Have the Demo Driver execute the final `./verify.sh 3` command to pass the corrected IAM JSON file into your Python evaluator.
- **How it works (Technically):** If your backend logic holds, the terminal will instantly flash the Game Over Screen. The UI will display the "THREAT NEUTRALIZED" banner, calculate the massive amount of fake money saved (e.g., \$42,100), and show the "Promoted to Senior SRE" status. This provides a natural, satisfying conclusion and a perfect applause line to end your 4-hour sprint.

Will you be the one manually driving the CLI commands during the live presentation, or will you focus entirely on delivering the narrative pitch to the judges while a teammate handles the terminal..??

User prompt: great now,, i want you to prepare a comprehensive md doc (which i will share to kiro,, which will help it to build/implement this project more precisely),, for this project which contains entire context of this project,, entire phases with their value and core and importance,, logic and answers of what, how, why, etc.,,, Everything about projecct,, end-to-end,, with all the detailed explanation of all possible questions of what, how, why...!!!

Response: I will now generate the comprehensive Master Specification Document for Kiro. This document is engineered specifically for an AI agent to consume, providing it with the precise architectural constraints, narrative context, file structures, and the "What, How, Why" logic necessary to execute your 4-hour hackathon sprint flawlessly.

User prompt: Great,, this project will be build in team of 2,, me and my friend,, so can you give me a step-by-step process we both can follow parallel to build it precisely and fast using Kiro in our individual laptops...!!

Response: This is the perfect scenario for a divide-and-conquer strategy. Since the project cleanly splits into two distinct domains—the **Python/LocalStack Backend** and the **Bash Terminal Theater**—you and your friend can work simultaneously on your individual laptops without stepping on each other's toes.

Here is your exact parallel execution roadmap for the 4-hour sprint.

Pre-Sprint: The 15-Minute Sync

Before the clock starts, do these three things:

1. **Create a GitHub Repo:** Create `iam-in-trouble` and invite your friend.
2. **Share the Kiro Spec:** Drop the `iam_in_trouble_spec.md` file (that we generated earlier) into the root of the repo so *both* of your local Kiro agents have the exact same master context.
3. **Assign Roles:**
 - **Developer A (The Backend Architect):** You will handle Python, boto3, and LocalStack.
 - **Developer B (The Terminal Director):** Your friend will handle Bash, ASCII art, and the live timer loop.

Hour 1 & 2: Parallel Kiro Sprints

Both of you will launch Kiro on your own laptops, feed it the Master Spec, and start prompting your specific domains.

Developer A (Backend / Python)

Environment: `docker run -d -p 4566:4566 localstack/localstack`

- **Prompt 1 (Setup):** "Kiro, read the master spec. Write `setup.py` using `boto3` to connect to LocalStack on port 4566. Have it spawn the `p4d` instance, the public S3 bucket, and write the broken `attacker_policy.json` to the local directory."
- **Prompt 2 (Verify Levels 1 & 2):** "Kiro, write `verify.py`. It should take a system argument (1, 2, or 3). For args 1 and 2, query LocalStack to ensure the EC2 instance is terminated and the S3 bucket is private."

- **Prompt 3 (Verify Level 3):** "Kiro, implement the custom Python JSON parser for arg 3. It needs to read `attacker_policy.json` and return a success code ONLY if the explicit Deny block is removed."
- **Your Manual Task:** Test the Python scripts by manually running AWS CLI commands to fix the environment, then running `python3 verify.py <level>` to ensure it catches the changes. Push to the backend Git branch.

Developer B (Terminal / Bash)

Environment: Standard Linux/WSL Terminal

- **Prompt 1 (The Engine):** "Kiro, read the master spec. Write `start_game.sh`. Generate a background bash loop that uses ANSI escape codes to print a ticking 'Bankrupt Counter' in the top right corner of the terminal that goes up by random dollar amounts. Make sure it runs asynchronously so the user can still type."
- **Prompt 2 (The Vibe):** "Kiro, write `briefing.sh`. Echo out a massive, neon-red ASCII art PagerDuty warning and the narrative text about the crypto miner."
- **Prompt 3 (The Wrapper):** "Kiro, write `verify.sh`. It should accept a level number, pass it to `python3 verify.py`, and if the exit code is 0, print a massive ASCII Victory Screen and `kill $!` to stop the background timer."
- **Your Manual Task:** Tweak the ASCII colors, ensure the background loop doesn't break the CLI input prompt, and push to the ui Git branch.

Hour 3: The "Merge & Wire" Phase

You both stop generating new features and integrate.

1. **Pull Everything:** Merge both branches into main. Both laptops should now have the Python backend scripts AND the Bash UI scripts.
2. **Wire them up:** Developer B (Bash) tests passing variables to Developer A's (Python) script.
 - *Test:* Run `./verify.sh 1`. Does it successfully call `python3 verify.py 1`? Does the Bash script correctly read the success/fail output and print the Victory screen?
3. **Squash Bugs:** LocalStack might take a second to update its mock state. You may need to ask Kiro to add a small 1-second `time.sleep()` in the verify scripts to prevent race conditions.

Hour 4: End-to-End Rehearsal

Put down the code. The final hour is strictly for running through the game as if you were the judges.

1. **The Full Reset:** Run `setup.py` to inject the broken state.
2. **The Live Run:** Run `./start_game.sh`.
3. **The Playthrough:** One of you frantically types the correct `aws --endpoint-url...` commands while the other reads the narrative out loud.
4. **Refinement:** If the game is too hard to complete in 3 minutes, or the money ticks up too slowly to be stressful, adjust the Bash loop variables.

By splitting the Python/AWS logic from the Bash/Visual logic, you entirely eliminate merge conflicts and double your effective Kiro generation speed. Who is taking the Backend, and who is taking the Terminal?

User prompt: give me the md doc for this,, and make sure to give detailed explanation of each step and detailed explained answer of all the what, why, how questions...!!!

Response: I will now generate the Parallel Execution Playbook. This document is designed for you and your teammate to reference throughout the hackathon. It breaks down exactly what each of you is responsible for, how to implement it using Kiro, and why the architecture is designed this way to survive a 4-hour time limit.

This playbook gives you a rigid, battle-tested structure for the entire 4 hours. You can print this out or keep it open on a second monitor while you work.

To maximize Kiro's effectiveness during the hackathon, you should literally copy the exact "Kiro Prompts" written under the "How" sections directly into your agent chats when the timer starts.

Are you and your teammate clear on who is taking the Backend (Developer A) and who is taking the Terminal (Developer B)?